Darren Lee
CSE13S

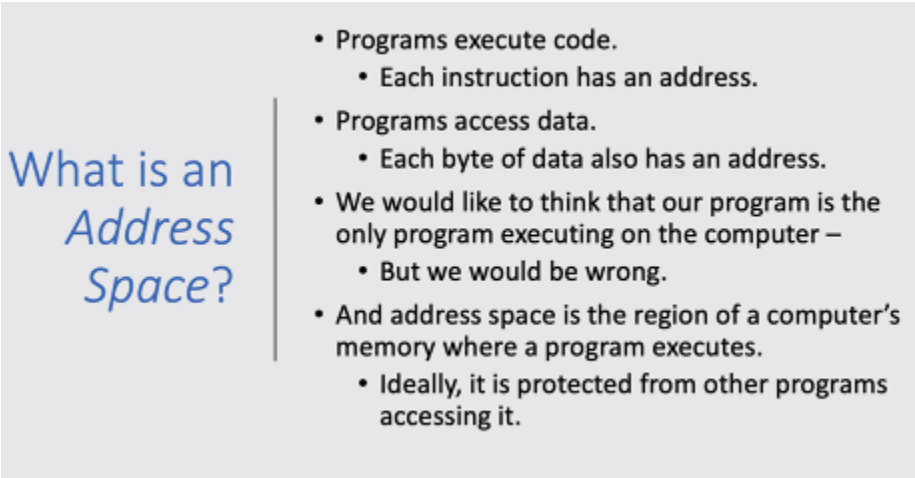
Week 7 Notes

5/10/21
Processes
Processes and threads

- Processes
- Threads
- Scheduling
- Inter-process communication (IPC)
- Classical IPC problems

What is a process?

- Code, data, and stack
    - usually (but not always) has its own address space
- Program state
    - CPU registers
    - Program counter (current location in the code)
    - Stack pointer
- Only one process can be running in a single CPU core at any given time
    - Multi-core CPUs can support multiple processes



How is that accomplished

- The loader could relocate the instruction by addressing a base address to each one
- There could be register that points to the first byte of program memory
- The processor and os could provide virtual memory

In an ideal world

- The ideal world has memory that is
    - Large
    - Fast
    - Non-volatile (doesn't go away when computer is turned off)
- The real world has memory that is:

        - Very large
        - Very fast
        - Affordable!
        - Pick any two
- Memory management goal: make the real world look as much like the ideal world as possible

Memory hierarchy

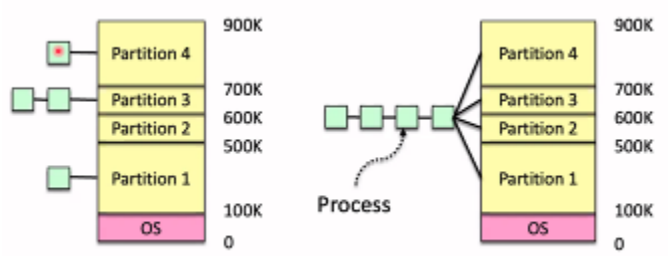
- Different levels of memory
- Some are small and fast
- others are large and slow
    - Cache: small amount of fast expensive memory
        - L1 cache usually on cPu chip
        - L2 may be on or off chip
        - L3 chase; off schip made of sram
    - Main memory: medium-speed, medium price memory (DRAM)
    - disk: many gigabytes of slow, cheap, non-volatile storage
- Memory manager handles memory hierarchy

Basic memory management

- Components include
    - Operating system
    - Single process
    - Goal: lay these out in memory
    - Memory protection may not be an issue (only one program)
    - Flexibility may still be useful *allow OS changes, etc)
    - No swapping or paging

Fixed partitions:

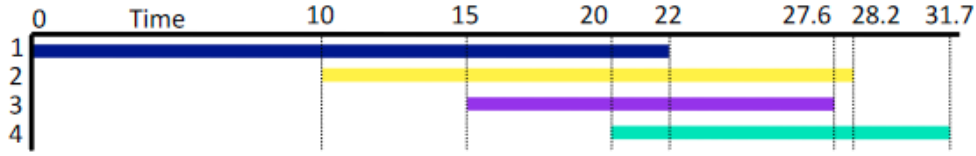
Multiple programs

- Fixed memory partition
    - Divide memory into fixed spaces
    - Assign a process to a space when it's free
- Mechanisms
    - Separate input queues for each partition
    - Single input queue: better ability to optimize cpu usage
    - 

Multiprogrammed system performance

Job	Arrival time	CPU needed
1	10:00	4
2	10:10	3
3	10:15	2
4	10:20	2

	1	2	3	4
CPU idle	0.8	0.64	0.51	0.41
CPU busy	0.2	0.36	0.49	0.59
CPU/process	0.2	0.18	0.16	0.15



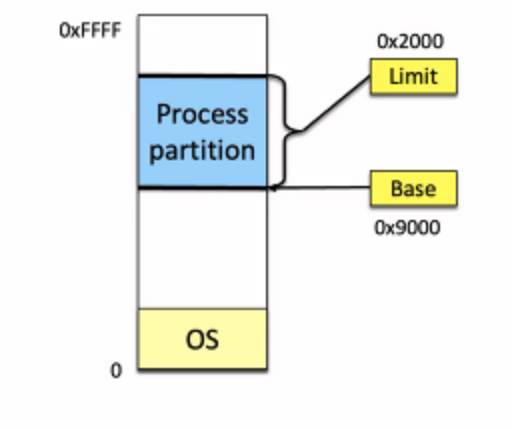
- Arrival and work requirements of 4 jobs
- CPU utilization for 1-4 jobs with 80% I/O wait
- Sequence of events as jobs arrive and finish
    - Numbers show amount of cpu time jobs get in each interval
    - More processes-> better utilization

Memory and multiprogramming

- Memory needs two things for multiprogramming
    - Relocation
    - Protection
- The os cannot be certain where a program will be loader in memory
    - Variable and procedures can't use absolute locations in memory
    - Several ways to guarantee this
- The os must keep processes' memory separate
    - Protect a process from other processes reading o modifying its one memory
    - Protect a process from modifying its one memory in undesirable ways *such as writing to program code)

Base and limit registers

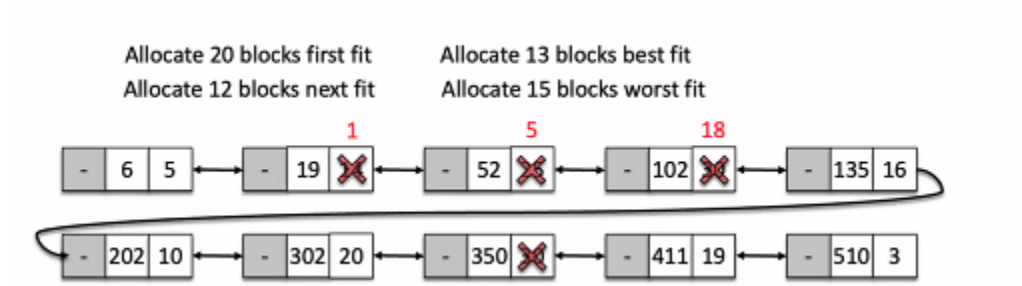
- Special cpu registers: base and limit
    - Access to the registers limited to system mode
- Register contain
    - Base: start of the process's memory partition
    - Limit: length of the process's memory partition
- Address generation
    - Physical address: location in actual memory
    - Logical address: location from the process's point of view
    - Physical address=base+logical address
    - Logical address larger than limit->error



Logical address: 0x1204
Physical address: 0x1204+0x9000 = 0xa204

Allocating memory

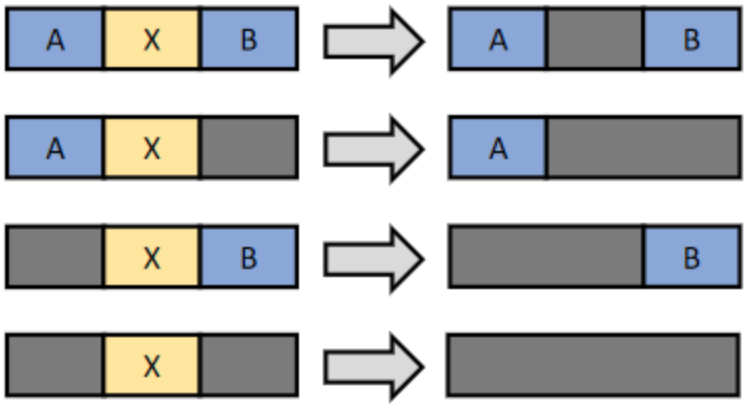
- Search through region list to find large enough space
- Suppose there are several choices: which one to use?
    - First fit: the first suitable hole on the list
    - Next fit: the first suitable after the previously allocated hole
    - Best fit: the smallest hole that is larger than the desired region (wastes least space)
    - Worst fit: the largest available hole *leaves largest fragment
- Option: maintain separate queues for different size holes



First fit is bad
Best fit is bad
Worst fit is better than best fit
Next fit is best on average

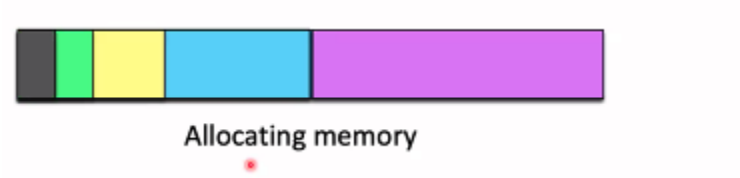
Freeing memory

- Allocation structures must be updated when memory is freed
- Easy with bitmaps: just set the appropriate bits in the bitmap
- Linked lists: modify adjacent elements as needed
    - Merge adjacent free regions into a single region
    - May involve merging two regions with the just-freed area



Buddy allocation

- Allocate memory in powers of two
  - Good for objects of varied sizes
- Split larger chunks to create two smaller chunks
- When chunk is freed, see if it can be combined with its buddy to rebuild a larger chuk
  - This is recursive

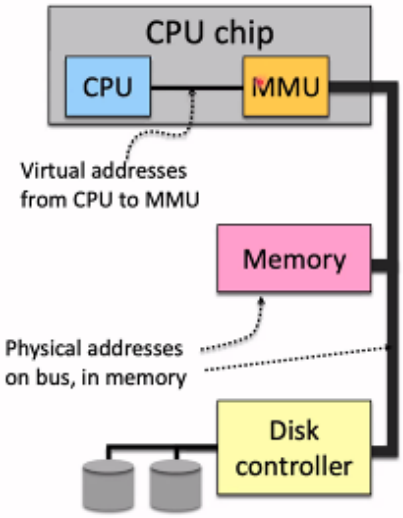


Virtual memory

- Basic idea llow the os to hand out more memory than exists on the system
- Keep recently used stuff in physical memory
- Move less recently used stuff to disk
- Keep all of this hidden from processes
  - Processes still see and address space from 0-max_addresss
  - Movement of information to and from disk handled by the os without process help
- Virtual memory (VM) especially helpful in multiprogrammed systems
  - Cpu schedules process B while process A watis for its memory to be retrieved from disk

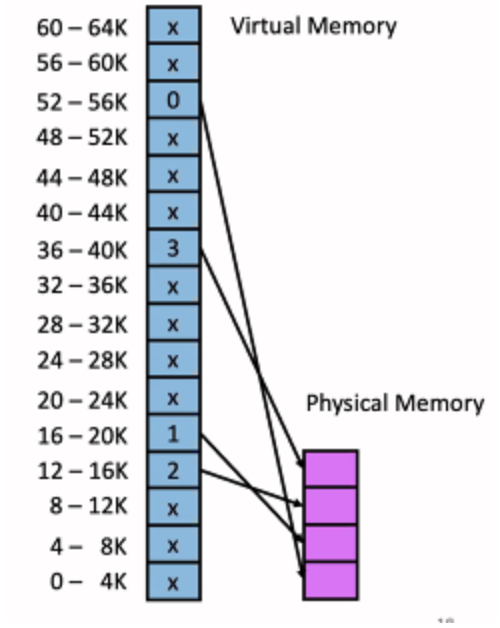
Virtual and physical addresses

- Program uses virtual addresses
  - Addresses local to the process
  - Hardware translates virtual address to physical address
- Translation done by the Memory Management Unit
  - Usually on the same chip as the CPU
  - Only physical addresses leave the CPU/MMU chip
- Physical memory indexed by physical addresses

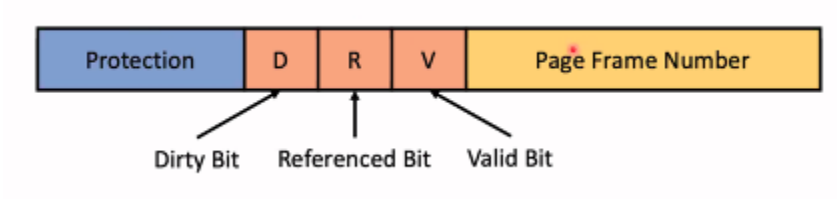


Paging and page tables

- Virtual addresses mapped to physical addresses
  - Unit of mapping is called a page
  - All addresses in the same virtual page are in the same physical page
  - Page table entry (PTE) contains translation for a single page
- Table translates virtual page number to physical page number
  - Nota ll virtual memory has a physical page
  - Not every physical page need to be used
- Example
  - 64 KB virtual memory
  - 16 KB physical memory



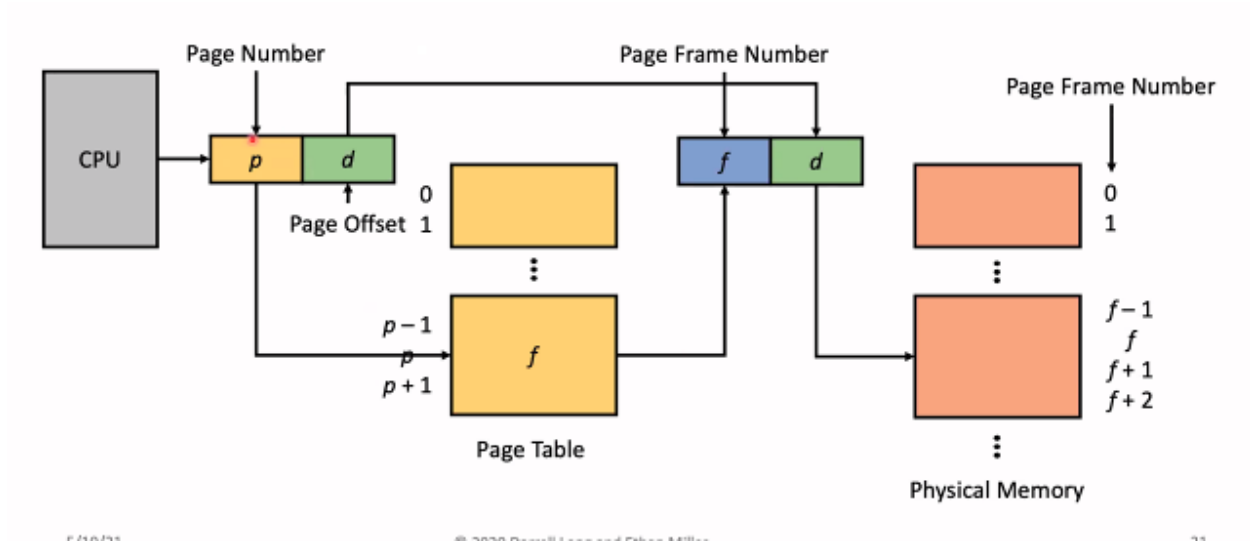
What's in a page table entry?

- Each entry in the page table contains
  - Valid bit: seti if this logical page number has a corresponding physical frma in memory
    - If not valid, remainder of PTE is irrelevant
  - Page frame number: page in physical memory
  - Referenced bit: set if adata on the page has been accessed
  - dirt(modified) bit: set if data on the page has been modified
  - Protection information
  - 

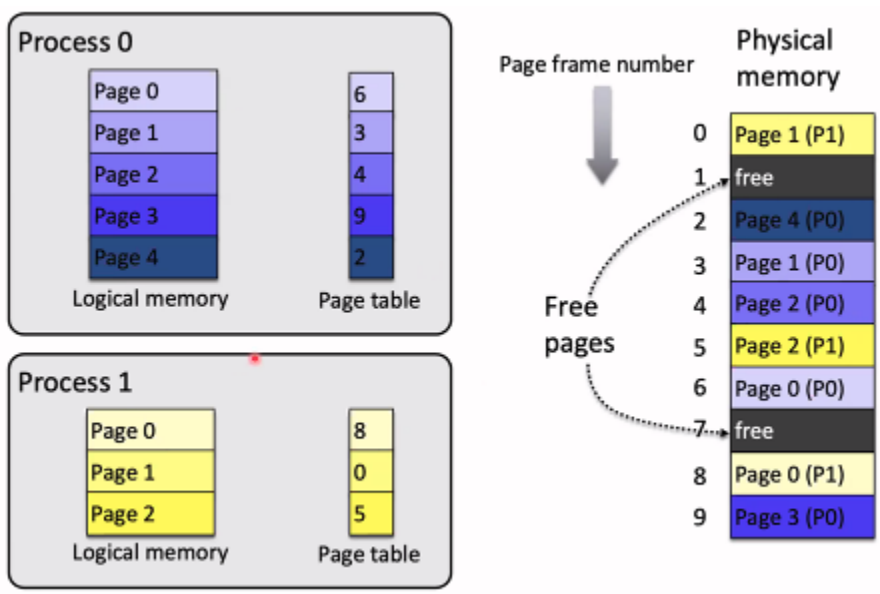
Mapping logical addresses to physical addresses

- Split address from CPU into two pieces
  - Page number (p)
  - Page offset (d)
- Page number
  - Index into page table
  - Page table contains base address of page in physical memory
- Page offset
  - Added to base address to get actual [physical memory address
- Page size=2^d bytes

Address translation architecture

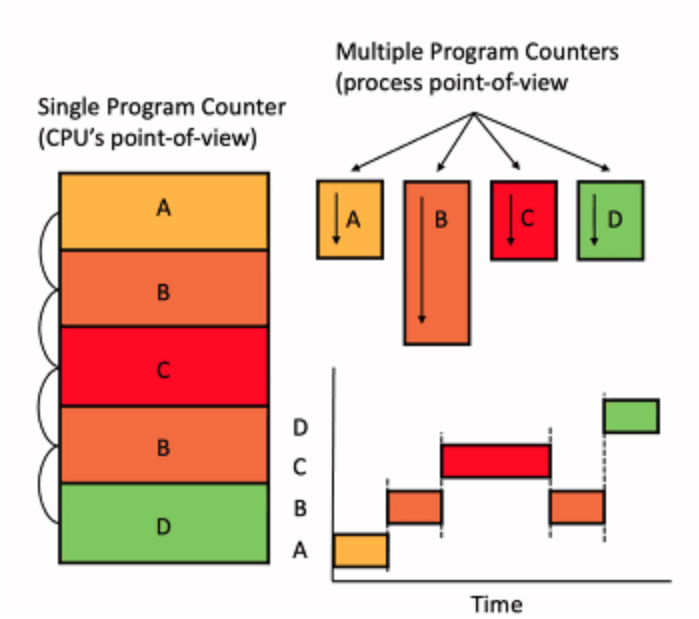


Memory and paging structures



The process model

- Conceptual model of multiprogramming four programs
  - 4 independent processes
  - Processes run sequentially
- Only on process active at any instant
  - An instant can be very short
  - Only applies if there's a single cpu with a single core in the system



When is a process created?

- Processes can be created in two ways
    - System initialization: one or more processes created when the os starts up
    - Execution of a process creation system call: something explicitly asks for a new process

- System call can come from
    - User request to create a new process (system call executed from user shell)
    - Already running process
        - User programs
        - System daemons

When do processes end

- Conditions that terminate process can be
    - voluntary
    - Involuntary
- Voluntary
    - Normal exit
    - Error exit
- Involuntary
    - Fatal error (only sort of involuntary)
    - Killed by another process

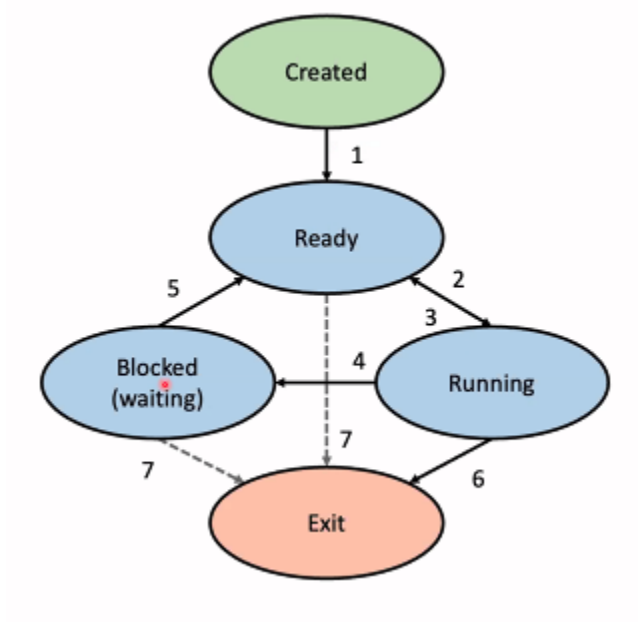
Process hierarchies

- Parent creates a child process
    - Child processes can create their own children
- Forms a hierarchy
    - Unix calls this a “process group”

    - If a process terminates, its children are inherited by the terminating process's parent
- Windows has process groups
    - Multiple processes grouped together
    - One process is the group leader

Process states

- Process in one of 5 states
    - Created
    - Ready
    - Running
    - Blocked
    - Exit
- Transitions between states
- 1. Process enters ready queue
- 2. Scheduler picks this process
- 3. Scheduler picks a different process
- 4. Process waits for an event such as I/O
- 5. Event occurs
- 6. Process exits
- 7. Process ended by another process



What's in a process table entry?

May be stored on stack

Process management

- Registers
- Program counter
- CPU status word
- Stack pointer
- Process state
- Priority / scheduling parameters
- Process ID
- Parent process ID
- Signals
- Process start time
- Total CPU usage

File management

- Root directory
- Working (current) directory
- File descriptors
- User ID
- Group ID

Memory management

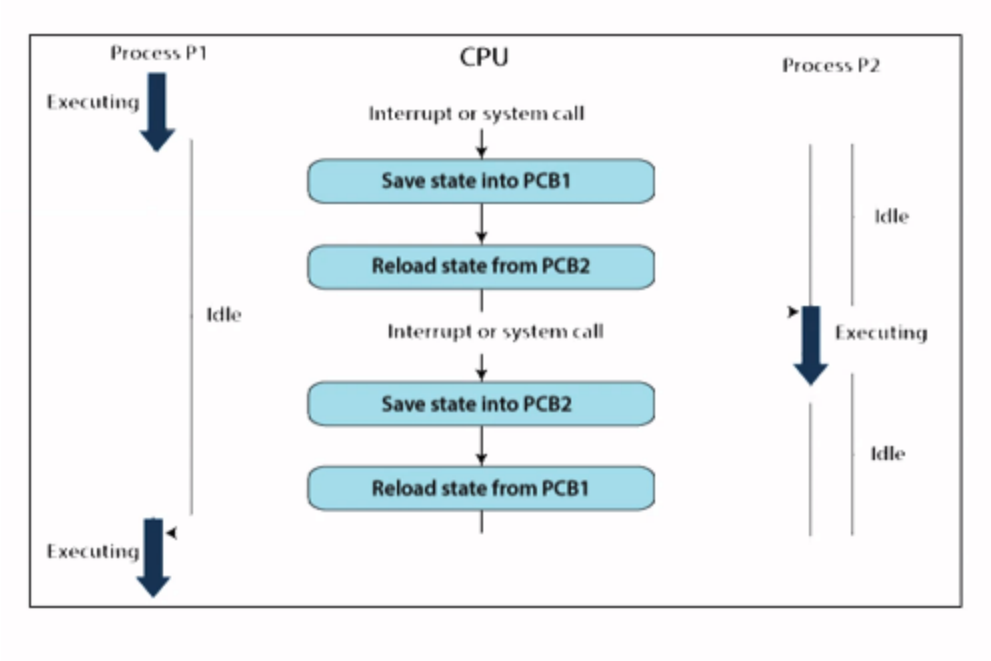
- Pointers to text, data, stack
- **- or -**
- Pointer to page table

```
  1  [                                        0.0%]   5  [|                                       0.7%]
  2  [                                        0.0%]   6  [|                                       0.7%]
  3  [                                        0.0%]   7  [                                        0.0%]
  4  [                                        0.0%]   8  [                                        0.0%]
  Mem[||||||||||||||||||||||||||||||941M/31.2G]   Tasks: 153, 246 thr; 1 running
  Swp[|                              2.50M/976M]   Load average: 0.03 0.03 0.00
                                                   Uptime: 13 days, 03:59:09

    PID USER      PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
2221610 darrell    20   0 10804  4228  3288 R  0.7  0.0  0:00.22 htop
    998 gdm        20   0 4157M  168M 92180 S  0.7  0.5 17:59.86 /usr/bin/gnome-shell
   1039 gdm        20   0 4157M  168M 92180 S  0.0  0.5  0:37.16 /usr/bin/gnome-shell
  87137 root       20   0  477M 60580 14828 S  0.0  0.2  0:29.84 /usr/libexec/fwupd/fwupd
   1180 gdm        20   0  453M  7444  6132 S  0.0  0.0  0:52.94 /usr/libexec/gsd-sharing
      1 root       20   0  165M 13328  8272 S  0.0  0.0  0:19.00 /sbin/init splash
    331 root       19  -1   98M 60768 59200 S  0.0  0.2  0:05.76 /lib/systemd/systemd-journald
    366 root       20   0 23732  7408  4112 S  0.0  0.0  0:10.22 /lib/systemd/systemd-udevd
    732 systemd-r  20   0 24024 11448  7372 S  0.0  0.0  3:07.70 /lib/systemd/systemd-resolved
    741 root       20   0  233M  9008  6576 S  0.0  0.0  0:19.86 /usr/lib/accountsservice/accounts-d
    814 root       20   0  233M  9008  6576 S  0.0  0.0  0:00.09 /usr/lib/accountsservice/accounts-d
    735 root       20   0  233M  9008  6576 S  0.0  0.0  0:21.16 /usr/lib/accountsservice/accounts-d
    736 root       20   0  2540   780   712 S  0.0  0.0  0:00.00 /usr/sbin/acpid
    739 avahi      20   0  9288  4288  3232 S  0.0  0.0  9:00.72 avahi-daemon: running [dmz.local]
    740 root       20   0  8364  4840  4432 S  0.0  0.0  0:00.01 /usr/lib/bluetooth/bluetoothd
    743 root       20   0  9412  3052  2784 S  0.0  0.0  0:01.79 /usr/sbin/cron -f
    747 messagebu  20   0  9140  6160  3856 S  0.0  0.0  1:13.82 /usr/bin/dbus-daemon --system --add
    813 root       20   0  536M 18256 14688 S  0.0  0.1  0:20.53 /usr/sbin/NetworkManager --no-daemo
    818 root       20   0  536M 18256 14688 S  0.0  0.1  0:26.78 /usr/sbin/NetworkManager --no-daemo
F1Help  F2Setup F3SearchF4FilterF5Tree  F6SortByF7Nice -F8Nice +F9Kill  F10Quit
```

How do we switch from one process to the other?

- Context switch
- Stop what we are doing
- Sav all our important information
    - Where? The stack
- Choose a process from a queue
- Load its important ifnor
- Start again

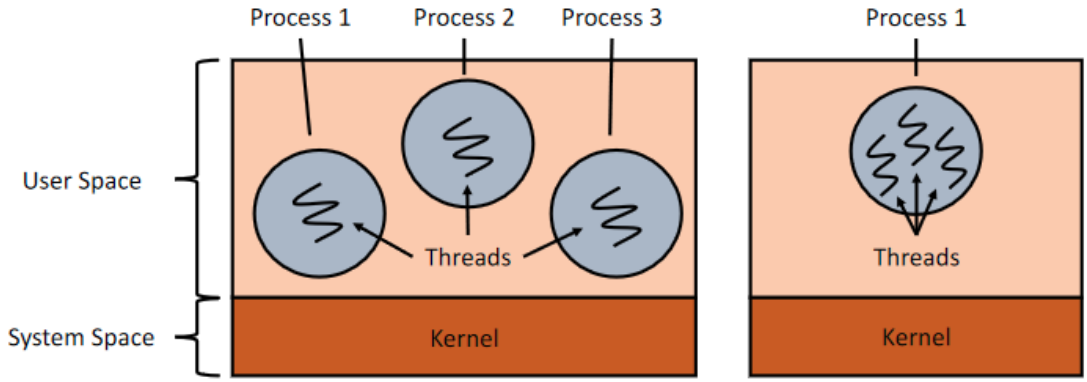


What happens on a trap/interrupt?

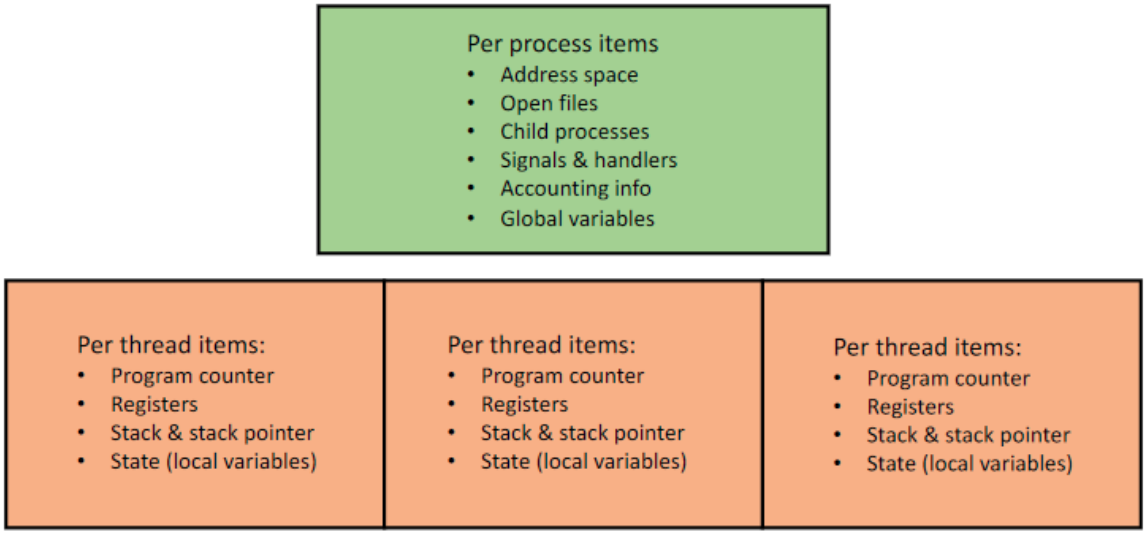
1. Hardware saves program counter (on stack or in a special register)
2. Hardware loads new PC, identifies interrupt
3. Assembly language routine saves registers
4. Assembly language routine sets up stack
5. Assembly language calls C to run service routine
6. Service routine calls scheduler
7. Scheduler selects a process to run next (might be the one interrupted)
8. Assembly language routine loads PC and registers for the selected process

Multiple threads in a process

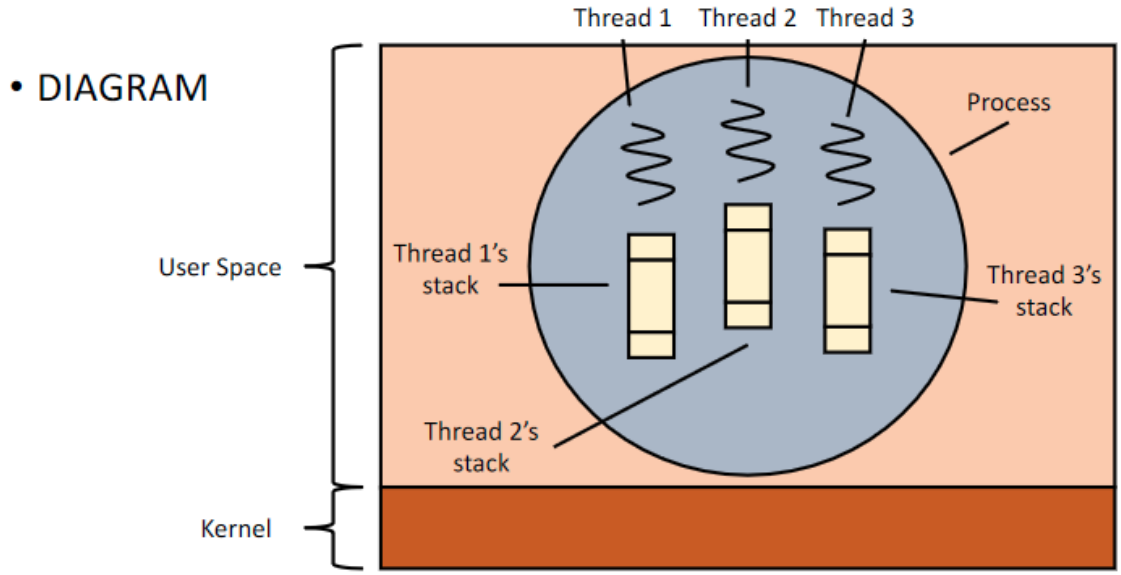
- DIAGRAM



Process and thread information



Threads and stacks



Why use threads?

- Allow a single application to do many things at once
  - Simpler programming model
  - Less waiting
- Threads are faster to create or destroy
  - No separate address space
- Overlap computation and I/O
  - Could be done without threads, but it's harder
- Example: word processor
  - Thread to read from keyboard

    - Thread to format document
    - Thread to write to disk

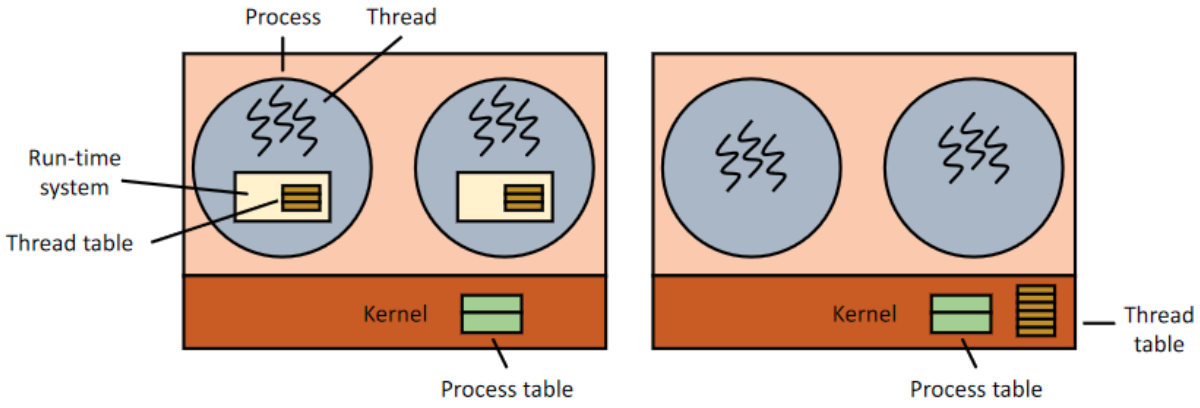
Three ways to build a server

- Multithreaded server
    - Parallelism
    - Blocking system calls
    - May use pop-up threads: create a new thread in response to an incoming message (rather than reusing a thread)
- Single-threaded process: slow, buyt easier to do
    - No parallelism
    - Blocking system calls
- Finite-state machine 9event model)
    - Each activity has its own state: states change when system calls complete or interrupts occur
    - Parallelism
    - Non Blocking system calls

Issues with using threads

- May be tricky to convert single-threaded code to multithreaded code
- Re-entrant code
    - Code must function properly when multiple threads are using it simultaneously
    - Need to be careful when using static or global variables
        - Returned structures
        - Buffers
- Error management
    - What happens when just a single thread has an error?
    - Can't simply kill the process, since other threads might be running

Implementing threads



POSIX threads

- Standard interface to threading library

- May be implemented in either user or kernel space
    - Some operating systems provide support for both
- Allows thread-based programs to be portable

Thread call (Pthread_xx)	**Description**
create	Create a new thread
exit	Terminate the calling thread
join	Wait for a specific thread to exit
yield	Release the CPU, allowing another thread to run

-

Processes and threads in Linux

- Supports POSIX standard
- Linux supports kernel-level threads (lightweight processes)
    - Share address space, file descriptors, etc.
    - Each has its own process descriptor in memory
- Linux processes (including lightweight) all have unique identifiers
    - Thread sharing address space are grouped into process groups
    - Identifier shared by the group is that of the leader
- Each process has its own 8KB region that stores
    - Kernel stack
        - Kernel has a small stack: about 4KB
- Other information stored in a separate data structure
    - Memory allocated to the process
    - Open files
    - Signal information

5/12/21
Innards of Language Translators

What is a language translator

- Simply put, a program that maps an input language to an output language

- Roles
    - Converting high-level source language to machine level target language
    - Provide diagnostic messages if the programmer violates specifications of the high-level language program

> *"John von Neumann, when he first heard about FORTRAN in 1954, was unimpressed and asked 'Why would you want more than machine language?' One of von Neumann's students at Princeton recalled that graduate students were being used to hand assemble programs into binary for their early machine. This student took time out to build an assembler, but when von Neumann found out about it he was very angry, saying that it was a waste of valuable scientific computing instrument to do clerical work."*
>
> —John A.N. Lee,
> Virginia Polytechnical Institute

The need for language translator

- Computers are a balanced mix of software and hardware
- Hardware functions are controlled by a compatible software
    - Binary characters, 0 and 1
    - Hardware codes are interpreted in binary format
    - Difficult and cumbersome task for computer programmers to write such codes

How do we compile a program?

```c
# include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

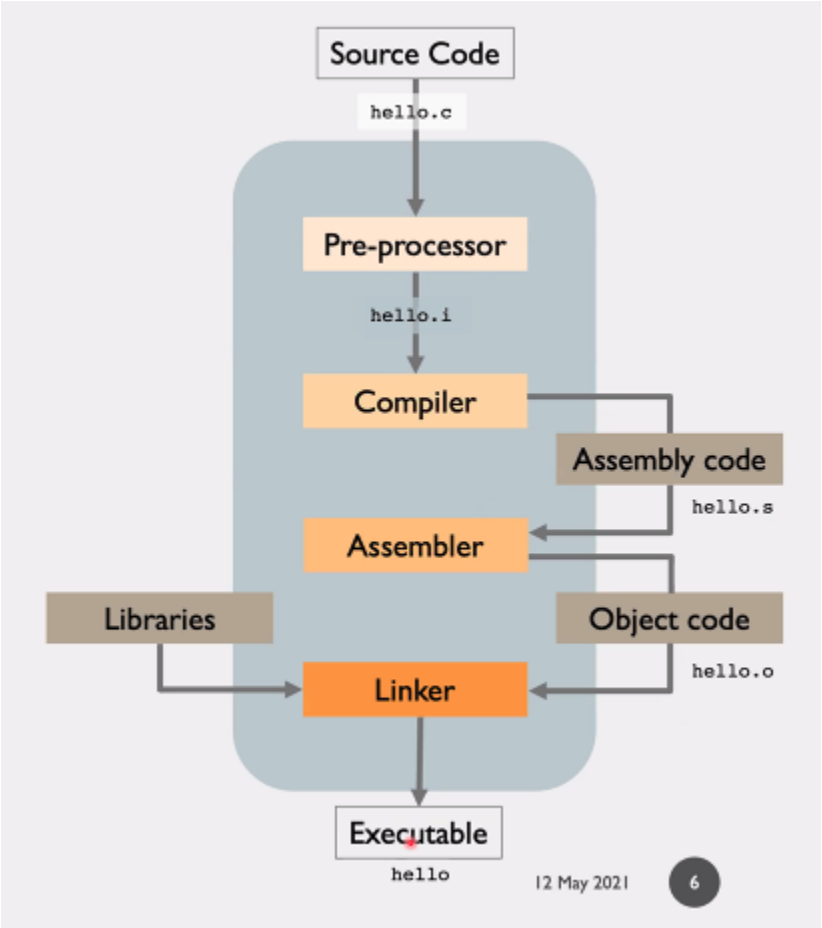
```asm
_main:                                  ## @main
    .cfi_startproc
## %bb.0:
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    leaq    L_.str(%rip), %rdi
    movb    $0, %al
    callq   _printf
    xorl    %ecx, %ecx
    movl    %eax, -8(%rbp)          ## 4-byte Spill
    movl    %ecx, %eax
    addq    $16, %rsp
    popq    %rbp
    retq
    .cfi_endproc
                                    ## -- End function
    .section    __TEXT,__cstring,cstring_literals
L_.str:                             ## @.str
    .asciz  "Hello World!\n"
```

Compilation process of a C program

The compiling process is a sequence of four phases the input for each phase is the output of the previous phase

1. Pre-processor

    a. Including macros, and the macro processor
2. Compiler
3. Assembler
4. Linker



Compilers: lexical phase

- Break down the source code from preprocessor into lexemes, generating a sequence of tokens as a unit in the grammar of the programming language
    - Tokens used to construct a parse tree in the next step of compilation
- Examples of tokens include keywords *int), identifiers, operators, and separators ((), {}, |)
    - Non-tokens: comments, preprocessor directive, macros, whitespace
    - Throws error if sequence does not match any valid token

Parse trees and e-t-f grammar
Completed during the syntax analysis phase
    Based on CFG (context free grammar) from language

- Parse tree generated, used to check if an expression is syntactically correct

- Example Grammar:

  S → E

  E → E + T

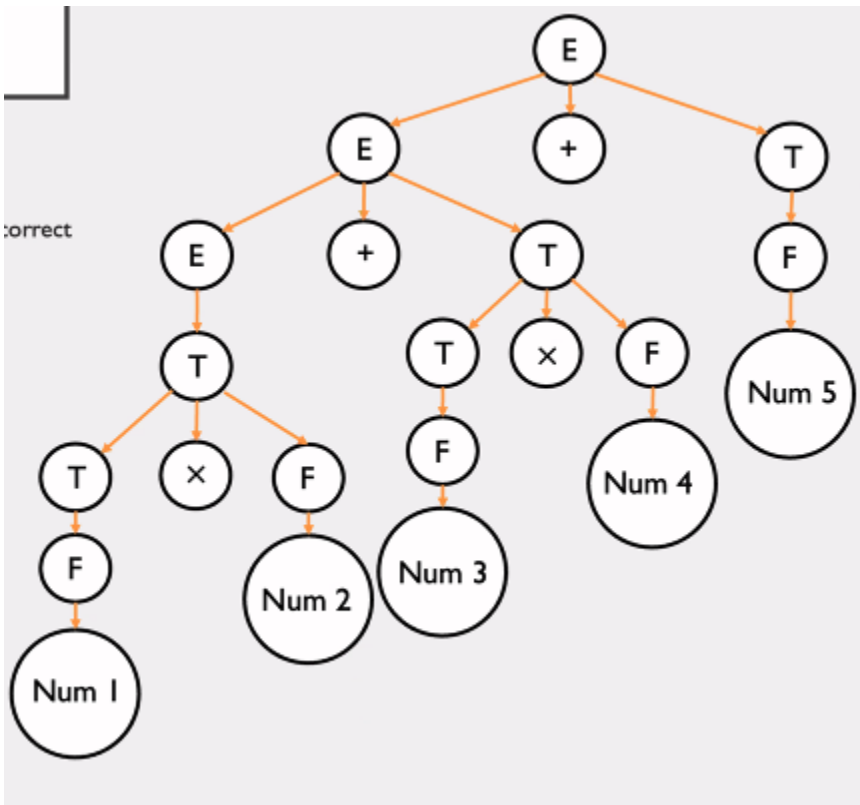
  | E – T

  | T

  T → T × F

  | T ÷ F

  | F

  F → (E) | Number | Variable
- Let's try parsing 1 × 2 + 3 × 4 + 5 into a tree
  - How would this look compared to the parse tree generated for (1 × 2) + 3 × (4 + 5)?
- 

Abstract syntax trees

- A representation of tokens generated from statements and expressions in the programming language
  - Represented as a hierarchical tree-like structure, similar to concrete syntax trees
  - Used for generating symbol tables and representing constructs and rules in the language
- Let's write an abstract syntax tree for the following code

3. The assembler
Converts assembly language instruction to machine language (binary)
Assemblers can perform conversion in one of two ways

- One pass: traverse through assembly once symbols used before being defined must inclined an errata at the end of the object code, letting the linker know to overwrite the undefined symbol with its respective definition
- Two pass; create a mapping of symbols and their values in first traversal , then replace symbols with their definitions upon second pass translation.

Forward referencing in assembly

- One pass forward referencing
    - Cannot resolve forward references, therefore, back-patching used to fill in spots of unknown address

4. The linker

- linke s hello.o along with any other object files and libraries
- Ensure that all dependencies are properly resolved
    - Linker error thrown if defitnitions to something cannot be referenced
- Merges everything into one executable file
    - In this case, hello

Loaders

- Lays in the operating system
- Ensures program and necessary libraries are place in RAM to prepare them for execution
- The loader has 4 basic functions
- Allocation - allocate space in memory fro program
- Linking -resolve symbolic references between program
- Relocation - fix all dependent locations and point them to the newly allocated space
- Loading - place machine code and data directly into the processor

Memory

- Memory layout of an executable can be seen in the figure on the right
- 1. Stack for local variables, return addresses, arguments and return values
- 2. Heap for dynamic memory allocation
- 3. Uninitialized data,

Preprocessor output

PREPROCESSOR OUTPUT

Notice how
`"//This is a comment!"`
has been stripped from the code by the preprocessor

```
int main(void)
{
 printf("Hello World!\n");
 return 0;
}
```

```
int asprintf(char ** restrict, const char * restrict, ...) __attribute__((__format__
(__printf__, 2, 3)));
char *ctermid_r(char *);
char *fgetln(FILE *, size_t *);
const char *fmtcheck(const char *, const char *);
int fpurge(FILE *);
void setbuffer(FILE *, char *, int);
int setlinebuf(FILE *);
int vasprintf(char ** restrict, const char * restrict, va_list)
__attribute__((__format__ (__printf__, 2, 0)));
FILE *zopen(const char *, const char *, int);




FILE *funopen(const void *,
                int (* _Nullable)(void *, char *, int),
                int (* _Nullable)(void *, const char *, int),
                fpos_t (* _Nullable)(void *, fpos_t, int),
                int (* _Nullable)(void *));
# 407 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/stdio.h" 3 4
# 1 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h" 1
3 4
# 31 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h"
3 4
# 1 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_common.h"
1 3 4
# 32 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h"
2 3 4
# 42 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h"
3 4
extern int __sprintf_chk (char * restrict, int, size_t,
      const char * restrict, ...);
# 52 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/secure/_stdio.h"
3 4
extern int __snprintf_chk (char * restrict, size_t, int, size_t,
      const char * restrict, ...);




extern int __vsprintf_chk (char * restrict, int, size_t,
      const char * restrict, va_list);




extern int __vsnprintf_chk (char * restrict, size_t, int, size_t,
      const char * restrict, va_list);
# 488 "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/stdio.h" 2 3 4
# 2 "hello.c" 2

int main(void)
{
 printf("Hello World!\n");
 return 0;
}
```

© 2020 Darrell Long

Compiler output

```
    .section    __TEXT,__text,regular,pure_instructions
    .build_version macos, 10, 15    sdk_version 10, 15
    .globl  _main                   ## -- Begin function main
    .p2align    4, 0x90
_main:                                  ## @main
    .cfi_startproc
## %bb.0:
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    leaq    L_.str(%rip), %rdi
    movb    $0, %al
    callq   _printf
    xorl    %ecx, %ecx
    movl    %eax, -8(%rbp)          ## 4-byte Spill
    movl    %ecx, %eax
    addq    $16, %rsp
    popq    %rbp
    retq
    .cfi_endproc
                                        ## -- End function
    .section    __TEXT,__cstring,cstring_literals
L_.str:                                 ## @.str
    .asciz  "Hello World!\n"


.subsections_via_symbols
```

Object file output

Compilers vs interpreters

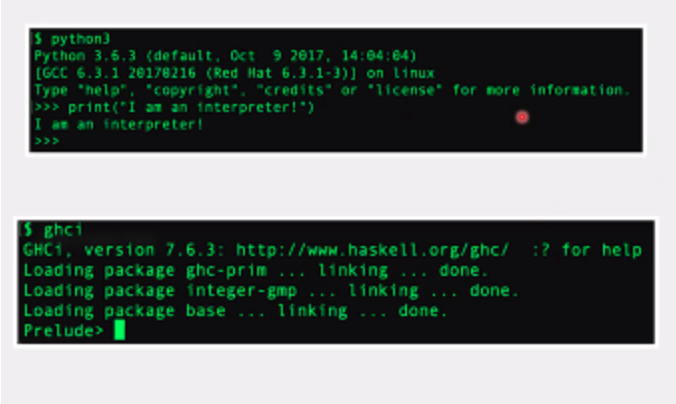
Compiler

- A translator that takes a high-level programming language, goes through a sequence of translations and outputs an executable
- Translates entire program/code at once

Interpreter

- A program that directly executes code without needing to compile (Python interpreter; ghci (Haskell))
- Translates program one line at a time

Which is faster, the compiler or an interpreter?

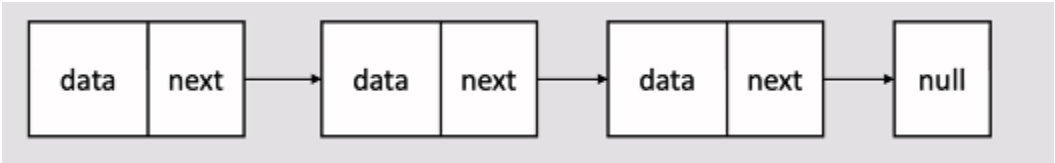


Gcc vs cc vs clang

- Gcc
  - Gnu c compiler
  - Default on linux
- Cc
  - unix/linux environment variable that points to the default compiler
- Clang
  - Default on mac and freebsd

5/14/21

Linked lists

- The diagram below depicts a singly linked list
- A linked data structure
  - In a singly linked list, each node contains a data field and a pointer to the next node in the list
  - In a double linked list, each node contains a data field and pointers to the next and previous nodes in the list
- The last node in the list points to a terminator, usually a null pointer
- 

Linked structures

- Linked lists are members of the class of linked structures
  - Linked lists
  - Trees
  - Tries
  - Graphs
  - Sparse matrices
  - …. And more

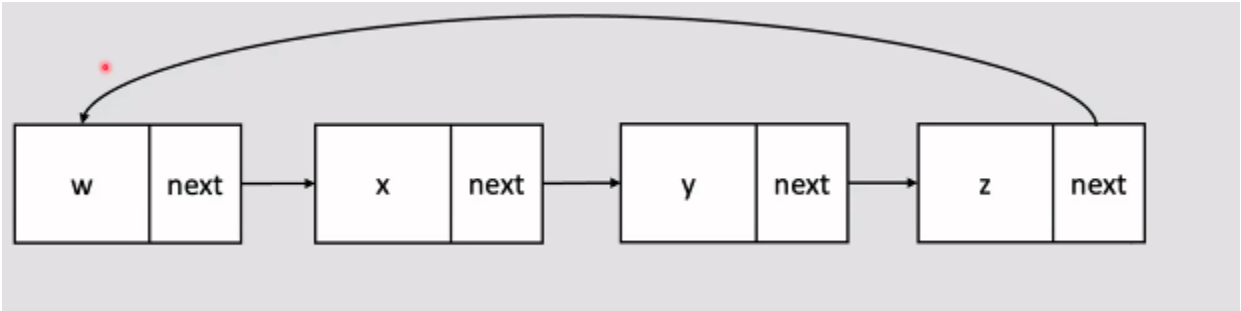
Advantages

- No fixed memory allocation
    - Grow and shrink at run-time without pre-allocating memory
    - No need to know the initial size of the list
- Insertion and deletions
    - No need to shift elements after insertion or deletion
    - Only update the address to the next pointer of a node
- Usage
    - Easily implement linear data structures like stacks and queues

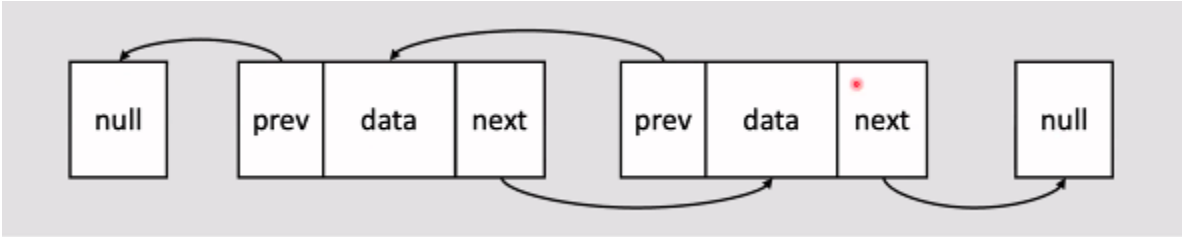
Disadvantages

- Memory usage:
    - Storing pointer to next node requires extra memory
    - Arrays are friendlier to processor caches
    - Slightly less memory efficient than arrays
- Traversal
    - Cannot randomly access elements, must traverse all elements up to the element we want to access
    - Reverse traversing is difficult in singly linked lists.
        - Easy in doubly linked list but uses extra memory to store an additional pointer

Circular singly linked list

- The last node of the linked list point back to the tail
- 

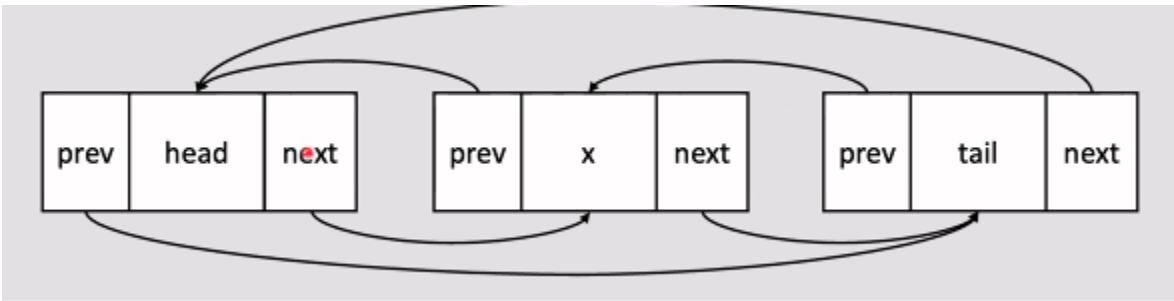
Doubly linked lists

- Each node has a pointer to both the previous and next nodes
- Allows traversal in two directions
- Less memory efficient than a normal linked list
- Typically implemented with sentinel nodes.
- 

Sentinel nodes

- Designated “dummy nodes” used to mark the ends of a linked list
- In a doubly linked list, sentinel nodes are placed at the head and tail
    - When performing an insertion, nodes will always go between two nodes
    - The grayed boxes below indicate the sentinel

Circular doubly linked list

- The head of the linked list ponts back to the tail
- The tail of the linked list points to the head
- 

A singly linked list ADT

```
typedef struct ListNode ListNode;

struct Listnode {
    char *key;
    ListNode *next;
};
```

Constructor

- Allocate memory needed for a single node
- A node’s key is the duplicated key
- A node initially points to NULL

```c
ListNode *node_create(const char *key) {
    ListNode *t = (ListNode *) malloc(sizeof(ListNode));
    if (t) {
        t->key = strdup(key);
        t->next = NULL;
    }
    return t;
}
```

© 2020 Darrell Long

Node destructor

- Free memory allocated for a node
- A double pointer is passed so we can null the original pointer

```c
void node_delete(ListNode **n) {
    if (*n) {
        free((*n)->key);
        free(*n);
        *n = NULL;
    }
    return;
}
```

© 2020 Darrell Long

Linked list destructor

- Walks the linked list and deletes each node

```c
void ll_delete(ListNode **head) {
    while (*head != NULL) {
        ListNode *next = NULL; // Save pointer to next node.
        next = (*head)->next;
        node_delete(head);
        *head = next;
    }
    return;
}
```

-

Lookup

- Walks the linked list to look for a specified key
    - If the key matches, then return the node, move on otherwise
- Linear search complexity for singly and doubly linked lists is O(n)
    - For keys that are strings with a maximum of m characters

Insertion

- Check if the key is already in the list
    - We don't want duplicates
- Create a node with the key
- Point the created node at the head
- The new head is the created node

```
ListNode *ll_insert(ListNode **head, const char *key) {
    if (ll_lookup(*head, key) != NULL) {
        return *head;
    }
    ListNode *n = node_create(key);
    n->next = *head;
    *head = n;
    return *head;
}
```

Printing

- Walk through the linked list and print out each node's key

```
void ll_print(ListNode *head) {
    for (ListNode *curr = head; curr != NULL; curr = curr->next) {
        printf("%s\n", curr->key);
    }
    return;
}
```

-

Removing

- Track the current and previous nodes
    - Initially, the previous node is null
- Walk the linked list
    - If the current node contains a matching key
        - If there was a previous node, point it at the node after the current node
        - Else, we were on the head, so make the head point to the node the current node is pointing at

        - Delete the found node, then we’re done
    - The previous node is now the current node
    - The current node is at its next node

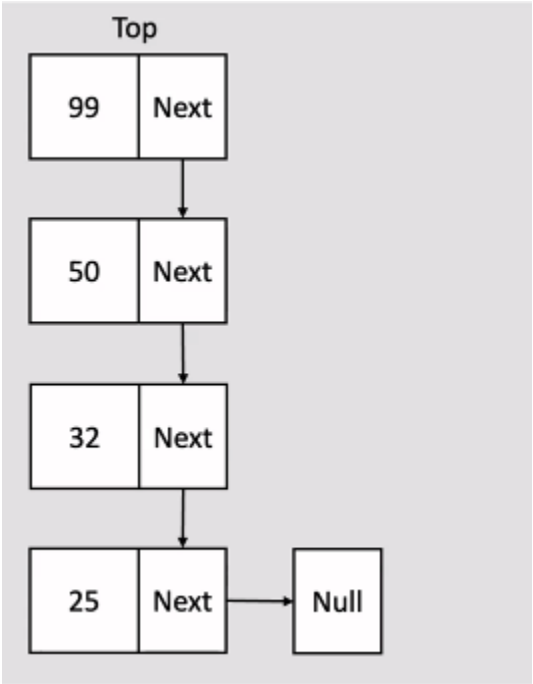
```c
ListNode *ll_remove(ListNode **head, const char *key) {
    if (*head) {
        ListNode *curr = *head;
        ListNode *prev = NULL;
        while (curr != NULL) {
            if (strcmp(curr->key, key) == 0) {
                if (prev != NULL) {
                    prev->next = curr->next;
                } else {
                    // If prev is NULL, we're on the head.
                    *head = curr->next;
                }
                node_delete(&curr);
                return *head;
            }
            prev = curr;
            curr = curr->next;
        }
    }
    return *head;
}
```

Move to front

- Track the current and previous nodes
    - Initially, the previous node is null
- Walk the linked list
    - If the current node contains a matching key
        - If there was a previous node, point it at the node after the current node
            - The current node should now point at the head
            - The new head is the current node, so we’re done
        - Else, we were on the head, so no need to move to the front
    - The previous node is now the current node
    - Current node is its next node

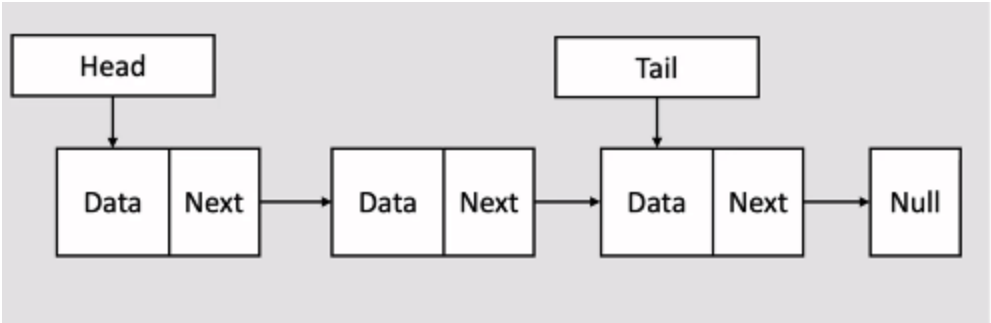
```c
ListNode *ll_mtf(ListNode **head, const char *key) {
    if (*head) {
        ListNode *curr = *head;
        ListNode *prev = NULL;
        while (curr != NULL) {
            if (strcmp(curr->key, key) == 0) {
                if (prev != NULL) {
                    prev->next = curr->next;
                    curr->next = *head;
                    *head = curr;
                    return *head;
                }
            }
            prev = curr;
            curr = curr->next;
        }
    }
    return *head;
}
```

Linked list stacks

- Stack size is limited only by available memory
- Pushing an element is inserting at the head
- Pooping an element is removing it from the head
- 

Linked list queues

- add at the tail
- Remove at the head



A doubly linked list adt

```
typedef struct listNode listNode;

struct listNode {
  char *key;
  listNode *fwd, *rev;
};

typedef struct {
  listNode *head, *tail;
} listHead;
```

Constructors

- To create a node
  - Allocate memory frot eh code
  - Duplicate the key for the node
  - The forward and reverse pointers are NULL to start with
- To create a list
  - Allocate memory for the list

```c
listNode *newNode(char *key) {
    listNode *n = (listNode *) malloc(sizeof(listNode));
    if (n) {
        n->key = strdup(key);
        n->fwd = n->rev = NIL;
    }
    return n;
}

listHead *newList(void) {
    listHead *h = (listHead *) malloc(sizeof(listHead));
    if (h) {
        h->head = h->tail = NIL;
    }
    return h;
}
```

Prepending

- Prepends a node n to the list (inserts at the head)
- If both the head and tail are NIL
    - The only node in the list is the node to prepeed
- Else
    - The node after n is the head
    - The node before the head is now n
    - The new head is n
    - There is no node behind n

```c
bool prependList(listHead *h, listNode *n) {
    if (h && n) {
        if (h->head == NIL && h->tail == NIL) {
            h->head = h->tail = n;
        } else {
            n->fwd = h->head;
            h->head->rev = n;
            h->head = n;
            n->rev = NIL;
        }
        return true;
    } else {
        return false;
    }
}
```

Appending

- Appends anode n to the list (inserts at the tail_)
- If both the head and tail are NIL
    - The only node in the list is sth en node to append
- Else
    - The node before n is the tail

```
bool appendList(listHead *h, listNode *n) {
    if (h && n) {
        if (h->head == NIL && h->tail == NIL) {
            h->head = h->tail = n;
        } else {
            n->rev = h->tail;
            h->tail->fwd = n;
            h->tail = n;
            n->fwd = NIL;
        }
        return true;
    } else {
        return false;
    }
}
```

Inserting

- Inserts a node n lexicographically
    - Specifically in reverse alphabetical order
- If both the head and tail are nil
    - The only node in th list in s the node to insert
- Else
    - Traverse to where the node should go
    - If we're at the end of the linked list. We append the node
    - If we're at the head of the linked list, we prepend the node
    - If we're in the middle
        - The current node is p
        - The node after n is the node p is point to
        - The node before n is now p
        - The node after p should point back to n
        - The node after p is now n

```c
bool insertList(listHead *h, listNode *n) {
    if (h && n) {
        if (h->head == NIL && h->tail == NIL) {
            h->head = h->tail = n;
        } else {
            listNode *p = h->head;
            while (p != NIL && strcmp(n->key, p->key) > 0) {
                p = p->fwd;
            }
            if (p == NIL || p == h->tail) {
                appendList(h, n);
            } else if (p == h->head) {
                prependList(h, n);
            } else {
                n->fwd = p->fwd;
                n->rev = p;
                p->fwd->rev = n;
                p->fwd = n;
            }
        }
        return true;
    } else {
        return false;
    }
}
```

```c
listNode *popList(listHead *h) {
    if (h && h->head) {
        listNode *p = h->head;
        h->head = p->fwd;
        p->fwd = p->rev = NIL;
        return p;
    }
    return NIL;
}
```

Popping

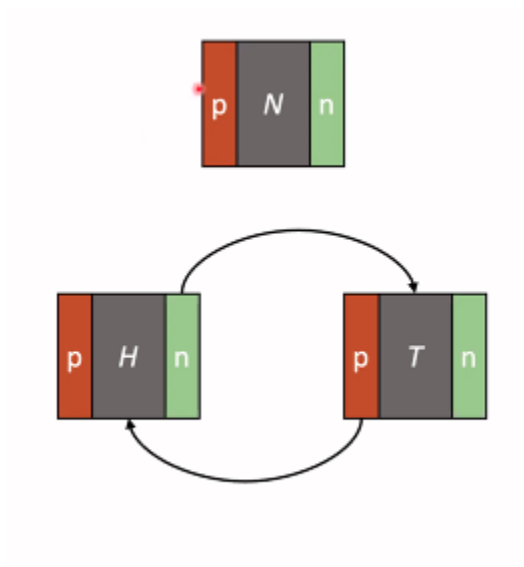
- Disconnects and returns the head of the linked list.
- If the head exists:
  - Save a pointer *p* to the head.
  - The new head is the node after *p*.
  - Make sure *p* doesn't point anywhere and return it.
- Else:
  - Return `NIL`.

Dropping

- Disconnects and returns that tail of the linked list
- If the tail exists
  - Save a pointer p to the tail
  - The new tail is the node before p
  - Make sure p does not point anywhere and return nil
- Else
  - Return nil

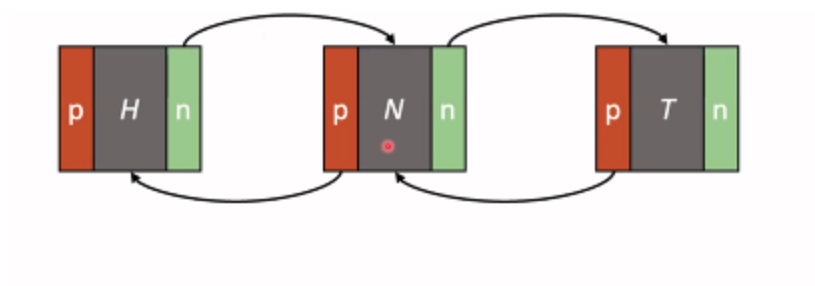
-
```c
listNode *dropList(listHead *h) {
    if (h && h->tail) {
        listNode *p = h->tail;
        h->tail = p->rev;
        p->fwd = p->rev = NIL;
        return p;
    }
    return NIL;
}
```

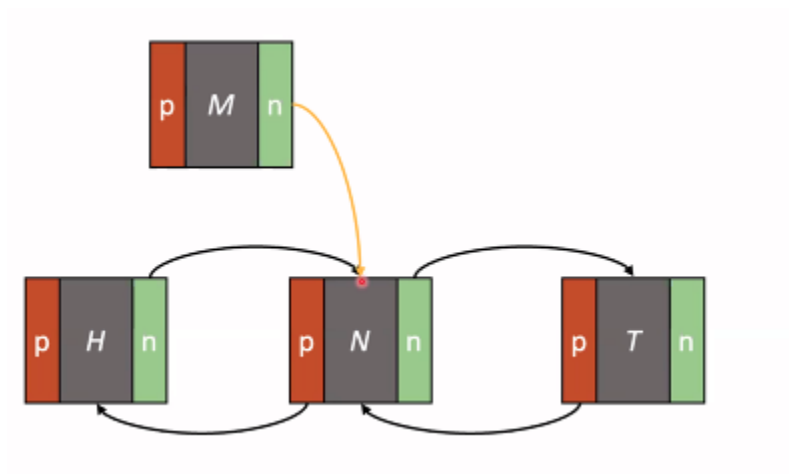
Insertion into a doubly linked list

- We have a new node N that we will insert
- 

    1. The node after n should be the node that h was pointing to
    2. The node before n should be the head sentient node h
    3. The node h is pointe to should now point back to n
    4. The node after h should now be n

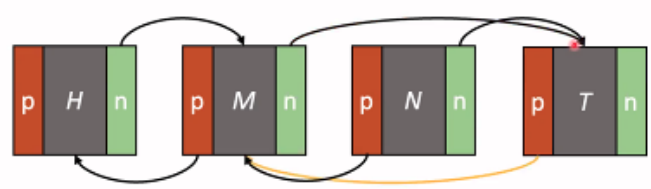
N is now at the front to ft he doubly linked list





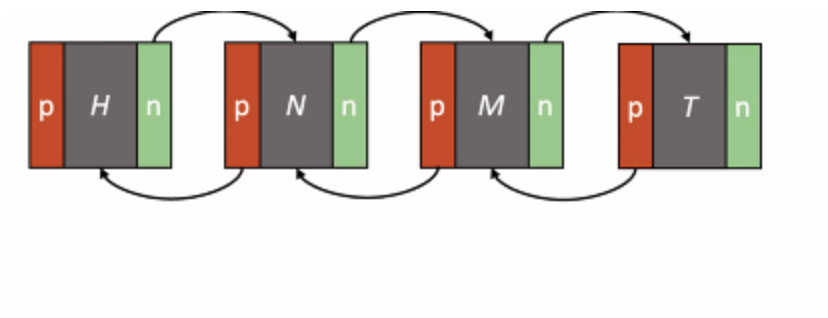
Move to front

1. The node before *N* should point to the node after *N*.
2. The node after *N* should point to the node before *N*.

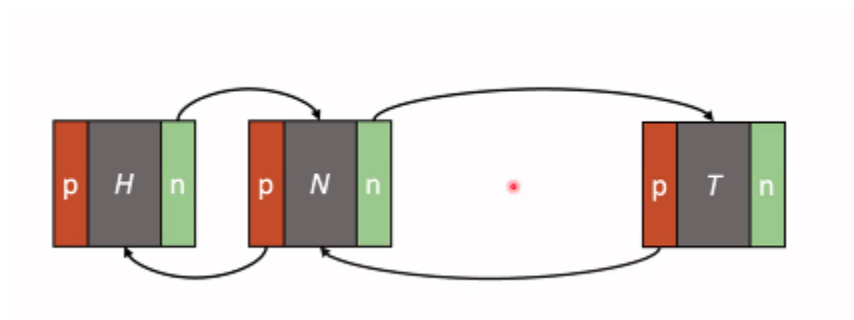


3. The node after N should be the node that the head sentinel node h awas pointing to
4. The node before n should now be h
5. The node after he should poinit back to n
6. The node after h should now be n

N is now at the front



Removing a node

- We decide the we don't like node m very much and want to remove it
- 1. The node before m should point to the node after m
- 2. The node after m should point to the node before m
- 3. Remove m

M is removed nwo